

Ingeniería del Software

Tema 9. Metodologías

TABLA DE CONTENIDOS

01

Características

02

Modelos de Evaluación

03

Comparativa

CARACTERÍSTICAS

Fruto de estas dos visiones han surgido multitud de metodologías. Cada metodología implementa el proceso de construcción de software prescribiendo un conjunto de roles, actividades y artefactos:

- ❑ **Roles** (Quién). Una persona puede desempeñar diversos roles, así como un mismo rol puede ser representado por varias personas. Un rol lleva cabo un conjunto de actividades y es “dueño” de un conjunto de artefactos.
- ❑ **Actividades** (Cómo). Las actividades tienen un objetivo concreto, normalmente expresado en términos de crear o actualizar algún artefacto.
- ❑ **Artefactos** (Qué). Es un trozo de información que es producido, modificado o usado por un proceso. Los artefactos o entregables son los resultados tangibles del proyecto.

CARACTERÍSTICAS

- ❑ Ejemplos de metodologías en cascada es la **ISO 12207** y su versión española **Métrica-3**.
- ❑ Ejemplos de metodologías ágiles son **Scrum** o **Kanban**.
- ❑ Existen también metodologías a medio camino entre ambos extremos como **RUP** (implementación de IBM del Proceso Unificado).

CARACTERÍSTICAS

Predictivos

Iterativos

Incrementales

Ágiles

Los requisitos son definidos por adelantado antes de que comience el desarrollo

Los requisitos pueden ser elaborados a intervalos periódicos durante la entrega

Los requisitos se elaboran con frecuencia durante la entrega

Entregar planes para el eventual entregable. Posteriormente, entregar solo un único producto final al final de la línea de tiempo del proyecto

La entrega puede ser dividida en subconjuntos del producto global

La entrega ocurre frecuentemente con subconjuntos del producto global valorados por el cliente

El cambio es restringido tanto como sea posible

El cambio es incorporado a intervalos periódicos

El cambio es incorporado en tiempo real durante la entrega

Los interesados clave son involucrados en hitos específicos

Los interesados clave son involucrados periódicamente

Los interesados clave son involucrados continuamente

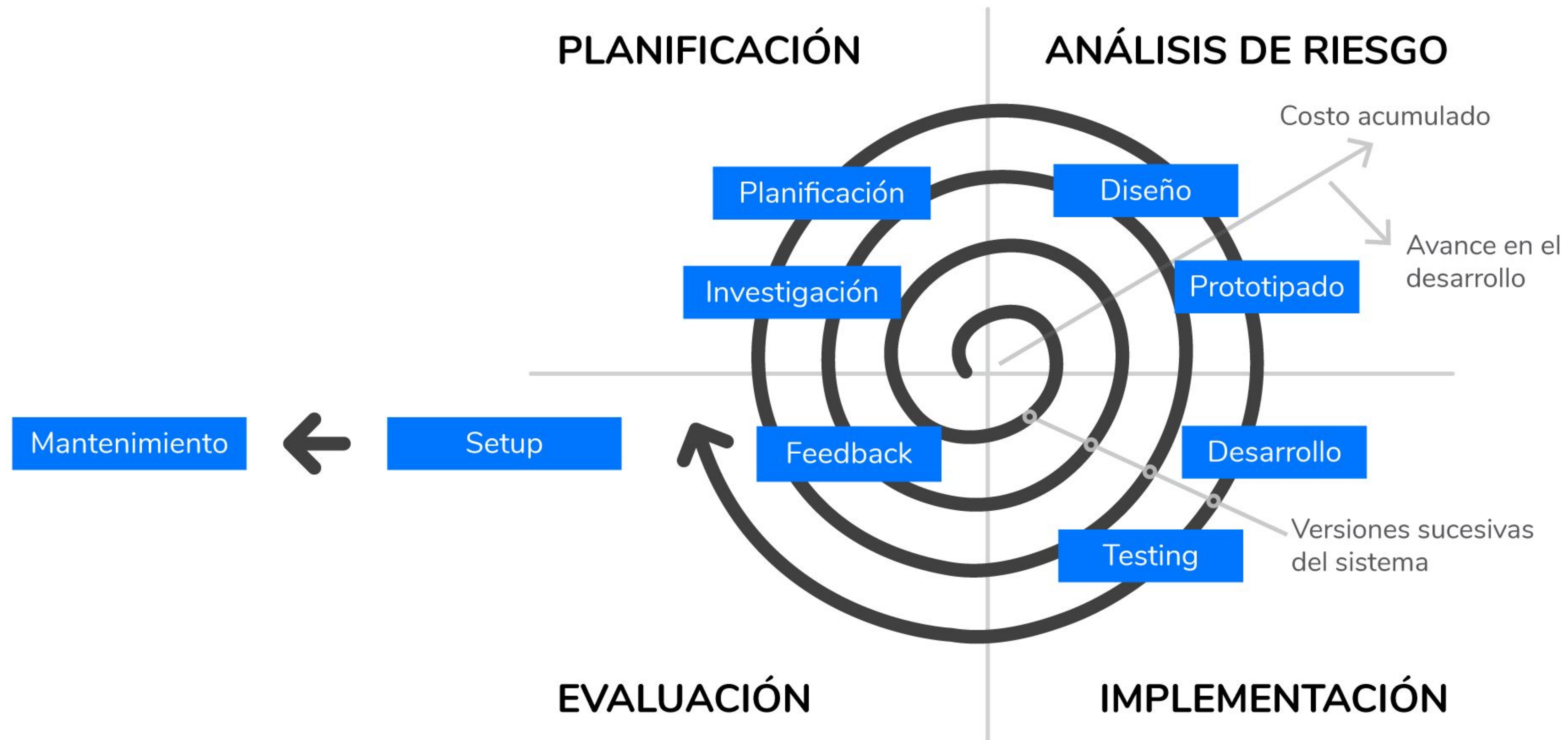
El riesgo y los costos son controlados mediante una planificación detallada de las consideraciones que mayormente se conocen

El riesgo y los costos son controlados mediante la elaboración progresiva de los planes con nueva información

El riesgo y los costos son controlados a medida que surgen los requisitos y limitaciones

MODELO EN ESPIRAL

El modelo en espiral fue propuesto por Barry Boehm en 1988. Puede considerarse un **refinamiento adaptativo del modelo en cascada**. El ciclo de iteración del modelo evolutivo se convierte en una espiral al añadir como dimensión radial una indicación del esfuerzo total realizado en cada momento.



METODOLOGIA RUP

El Proceso Unificado (UP) nace en 1999, intentando adaptarse a las peculiaridades de la industria del software dentro de la visión predictiva.

De Objectory a UP	
1967	Con la Metodología Ericsson (Ericsson Approach) elaborada por Ivar Jacobson, una aproximación de desarrollo basada en componentes, que introdujo el concepto de Caso de Uso
1987	Jacobson fundó la compañía Objectory AB y lanza el proceso de desarrollo Objectory (abreviación de Object Factory)
1995	Rational Software Corporation adquiere Objectory AB y entre 1995 y 1997 se desarrolla Rational Objectory Process (ROP) a partir de Objectory 3.8 y del Enfoque Rational (Rational Approach) adoptando UML como lenguaje de modelado.
1998	Rational Software Corporation lanza Rational Unified Process
1999	Ivar Jacobson, Grady Booch y James Rumbaugh publican “ <i>The Unified Software Development Process</i> ”. Nace el Proceso Unificado (UP).
2003	IBM adquiere Rational Software Corporation

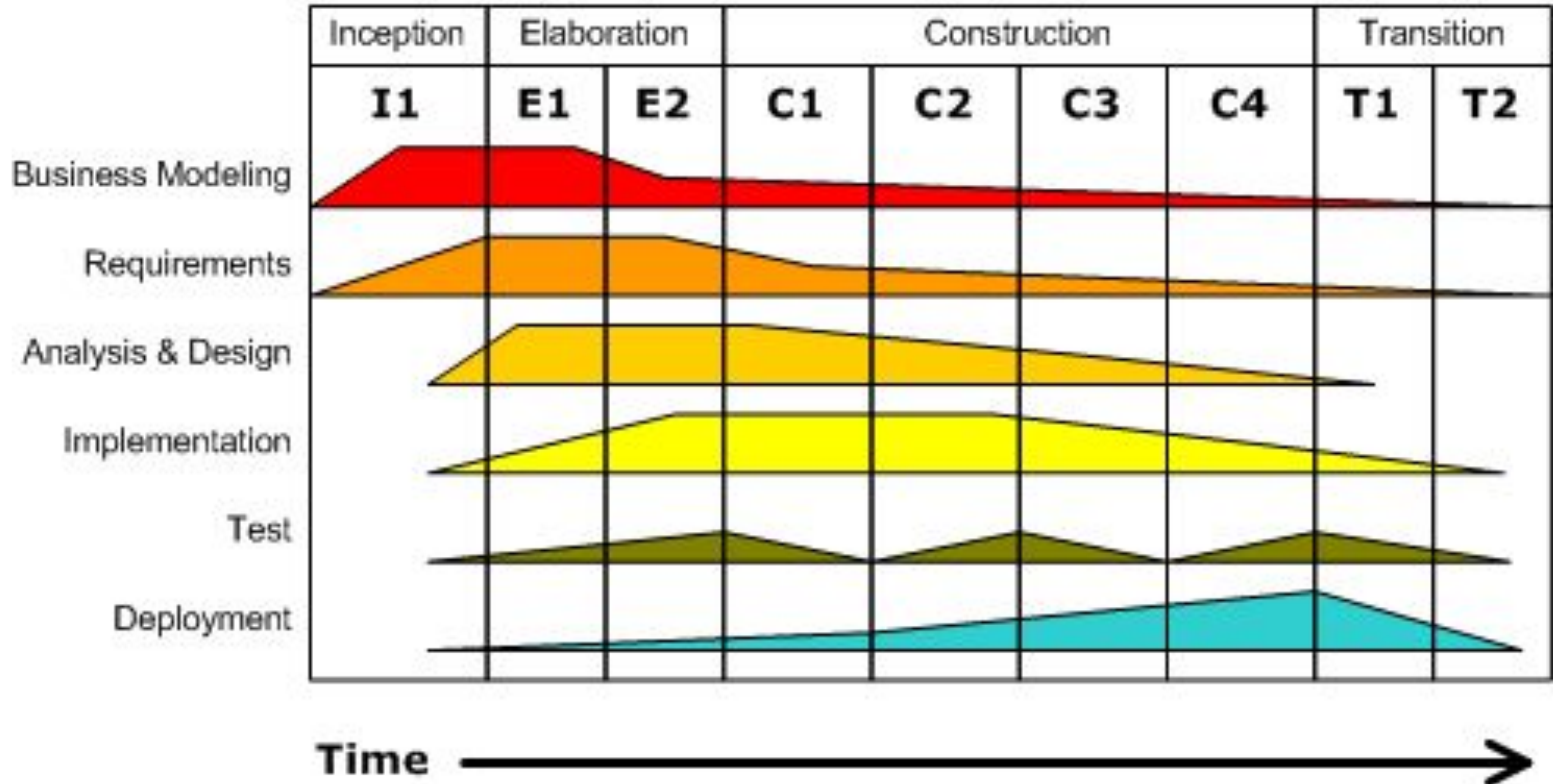
Así, la implementación más conocida del Proceso Unificado, Rational Unified Process (RUP), permite y acepta el solapamiento entre fases (requisitos, diseño, codificación, pruebas). El valor de negocio es desarrollado incrementalmente en iteraciones de duración determinada e interdisciplinarias.

RUP se describe desde tres perspectivas:

- ❑ Una perspectiva dinámica que muestra las fases del modelo a través del tiempo.
- ❑ Una perspectiva estática que fija las actividades del proceso.
- ❑ Una perspectiva práctica que sugiere buenas prácticas a usar durante el proceso

Iterative Development

Business value is delivered incrementally in time-boxed cross-discipline iterations.



METODOLOGIA RUP

RUP es una metodologia:

- ❑ Rígida, prescribe muchos roles, artefactos y tareas
- ❑ Iterativa e incremental, acepta los cambios

Scrum prescribe únicamente tres roles:

- ❑ **Product owner.** Es la voz del cliente. Debe asegurarse que el equipo aporta valor al negocio. El product owner debe centrarse en el aspecto de negocio del proyecto y por tanto pasar la mayor parte del tiempo actuando de intermediario entre el equipo y el cliente.
- ❑ **Equipo Scrum.** Es el responsable de entregar al final de cada sprint el desarrollo incremental previsto. El equipo scrum consta de entre 3 y 9 personas para ser eficiente. Esto incluye a quienes se encargan de analizar, diseñar, desarrollar, probar, documentar, etc.
- ❑ **Scrum Master.** El Scrum es facilitado por un Scrum Master, cuyo trabajo primario es eliminar los obstáculos que impiden que el equipo alcance el objetivo del sprint. El Scrum Master es el que hace que las reglas se cumplan.

METODOLOGIA SCRUM

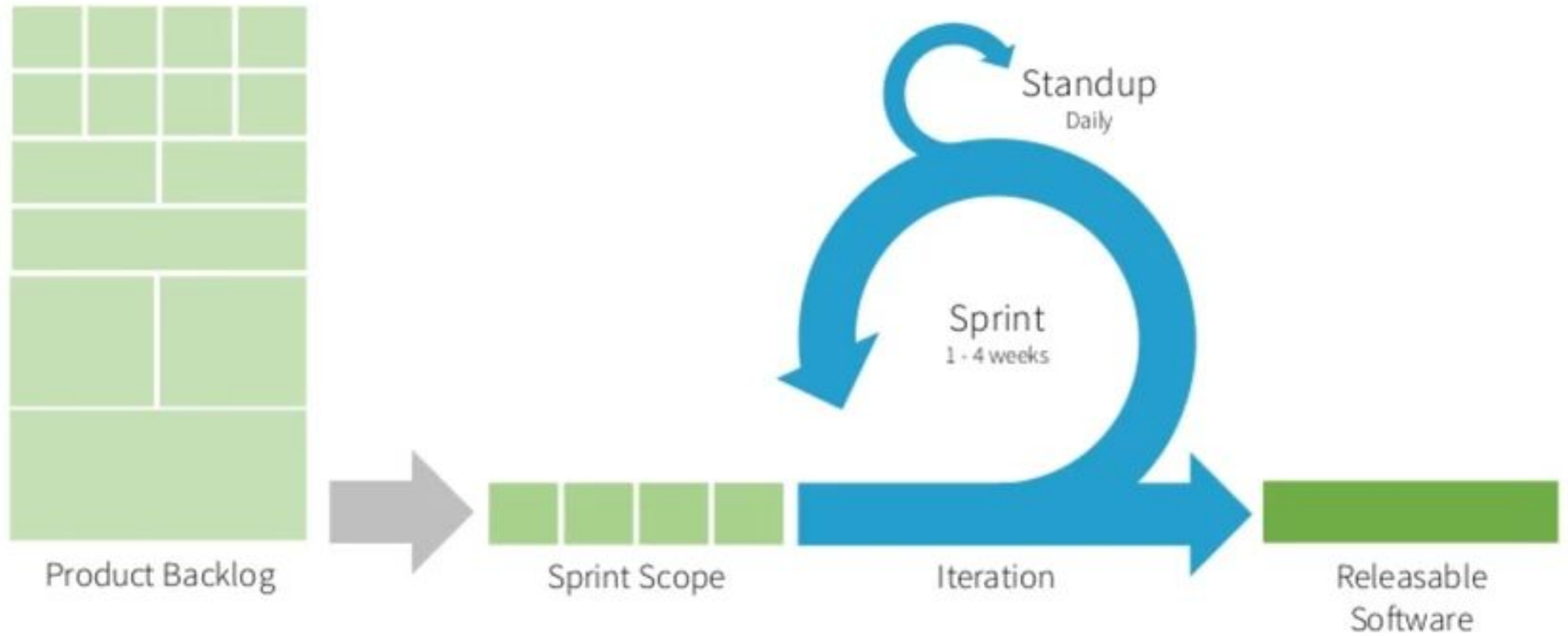


TABLA DE CONTENIDOS

01

Características

02

Modelos de Evaluación

03

Comparativa

MODELOS DE EVALUACIÓN DEL PROCESO

- ❑ Existen modelos para la mejora y **evaluación** de proceso de construcción de software implantado en la organización o empresa.
- ❑ Los modelos de evaluación más conocidos son CMMI y ISO 15504 SPICE.
- ❑ Es posible con estos modelos evaluar tanto metodologías predictivas como adaptativas.

MODELO DE EVALUACIÓN: CMMI



MODELO DE EVALUACIÓN: CMMI

	Área de proceso	Categoría	Nivel
1	Planificación de proyecto (PP)	Gestión de proyectos	2
2	Seguimiento y control de proyecto (PMC)	Gestión de proyectos	2
3	Gestión y acuerdo con proveedores (SAM)	Gestión de proyectos	2
4	Gestión de requisitos (REQM)	Ingeniería	2
5	Gestión de la configuración (CM)	Soporte	2
6	Medición y análisis (MA)	Soporte	2
7	Gestión calidad procesos y productos (PPQA)	Soporte	2
8	Definición de procesos organizacionales (OPD)	Gestión de procesos	3
9	Enfoque a Procesos de la organización (OPF)	Gestión de procesos	3
10	Formación Organizacional (OT)	Gestión de procesos	3
11	Gestión integral de proyecto (IPM)	Gestión de proyectos	3
12	Gestión de riesgos (RSKM)	Gestión de proyectos	3
13	Integración de producto (PI)	Ingeniería	3
14	Desarrollo de requisitos (RD)	Ingeniería	3
15	Solución técnica (TS)	Ingeniería	3
16	Validación (VAL)	Ingeniería	3
17	Verificación (VER)	Ingeniería	3
18	Análisis y resolución de decisiones (DAR)	Soporte	3
19	Rendimiento de los procesos de la organización (OPP)	Gestión de procesos	4
20	Gestión cuantitativa de proyectos (QPM)	Gestión de proyectos	4
21	Innovación y desarrollo Organizacional (OID)	Gestión de procesos	5
22	Análisis y resolución de problemas (CAR)	Soporte	5

MODELO DE EVALUACIÓN: ISO 15504 SPICE

- ❑ Determinación de la Capacidad de Mejora del Proceso de Software (SPICE) es una norma internacional para evaluar los procesos de las organizaciones. La ISO 15504 es un “framework” para evaluar de manera general cualquier modelo de procesos (sea relativo a la construcción de software o no).
- ❑ Califica los procesos en 6 niveles de madurez.
- ❑ Existen dos esquemas que adaptan la ISO 15504 al desarrollo de software, uno es el modelo impulsado principalmente por varias organizaciones de Reino Unido, el Pathfinder, y otro el de AENOR (Asociación Española de Certificación). Ambos esquemas toman como referencia para su evaluación los procesos definidos en la ISO 12207.

METODOLOGIAS: RUP VS SCRUM



TABLA DE CONTENIDOS

01

Características

02

Modelos de Evaluación

03

Comparativa

CONCLUSIÓN: RUP VS SCRUM

RUP Artifact	Comments	Format / Tools
Requirements		
Vision Document	Describes project objectives, important classes of users, and all product features. Features are uniquely identified for reference from other artifacts.	Text document, 15 pages
Use Case Model	Describes all use cases of the system in one document. Use cases were used to better understand the system, but not for project planning. Initial versions of the document included sketches of the GUI, in later versions the sketches were replaced with screen shots of the actual GUI.	Text document, 20 pages
Analysis and Design		
Software Architecture Document	Describes the high level software design of the system. Contains some UML diagrams plus brief descriptions of important mechanisms.	Text document, 12 pages
Design Model	Detailed software design of the system in terms of classes and objects and their grouping into packages. We used the CASE tool „Together / J“ for object modeling. This tool keeps design information as special comments in source code.	Together / J
Implementation		
Implementation Model	In RUP terminology the implementation model is simply the collection of all artifacts required to build the system, i.e. all source files, makefiles, configuration files, etc.	Java source code files, Makefiles, JBuilder
Test		
Defect List	A list of all open and closed defects, including a brief description of each defect.	Text document, 3 pages
Deployment		
Release Notes	Were written for each release given to the customer.	Text document, 2 pages
Installation Artifacts	These are all executable files, configuration files, installation procedures, documentation etc. needed to install the system. For each release a set of installation artifacts was created and given to customer.	ZIP file
Configuration and Change Management		
Configuration Management Plan	Describes the policies for version control, release management, and change request management. We used CVS for version and release management. All artifacts of the project were placed under version control.	Text document, 8 pages
Change Request List	A list with all open and closed change requests, including a very brief description of each change request.	Text document, 4 pages
Project Management		
Software Development Plan	A coarse grained plan with a list of all planned iterations. For each iteration, the major objectives, start and end dates were specified. This plan was updated after each iteration.	Text document, 8 pages
Iteration Plan	A detailed plan for each iteration, outlining which features, change requests, bug fixes and additional work items will be done by whom.	Text document, 4-6 pages (one for each iteration)
Iteration Assessment	A summary of the results of an iteration, i.e. what objectives were reached, what features, change requests and bug fixes were actually implemented, and reasons for any deviations from the plan.	Text document, 2-5 pages (one for each iteration)
Environment		
Development Case	Describes how RUP is adapted for a project. Mostly a list of which artifacts are used and which aren't.	Text document, 6 pages
Programming Guidelines	We reused company internal Java programming guidelines.	Text document, 10 pages

RUP prescribe no menos de diez artefactos, cada uno acompañado de, al menos, una actividad y un rol. SCRUM es menos rígida, más flexible y mucho menos prescriptiva.

CONCLUSIÓN: RUP VS SCRUM

*“Sólo ha habido dos ocasiones en que gente se uniese para discutir qué es el desarrollo software y qué hace que funcione, la primera en 1968 (conferencia de la OTAN) y la otra en 2001. **Las conferencias del 1968 y 1969 fueron un desastre para la historia y nos enviaron por el camino equivocado durante 30 años.** El manifiesto ágil del 2001 hizo un trabajo mucho mejor diciendo lo que funciona y por qué, en pocas palabras. Piensa en el Manifiesto Ágil diciendo: «podría haber 200 cosas importantes a las que prestar atención a un proyecto, es decir demasiadas. **Aquí están las 4 cosas a tener en mente a medida que avance, aquí hay otras 12 cosas a mirar**». Si lo ven de esta manera, el manifiesto tiene mucho sentido, pero también deja muchas opciones abiertas a otras formas de trabajar. Me resulta muy satisfactorio que los valores y los principios del manifiesto suenen todavía diez años después, y parece que van a sonar por un tiempo muy largo. Mientras el mundo interioriza esos valores y principios, como es natural, las variaciones, nuevas extensiones nuevas, nuevas palabras, serán más pertinentes para las nuevas generaciones. Pero esas serán las extensiones, no cambios fundamentales.”*

Alistair Cockburn



COMPARACIÓN ENTRE VISIONES

Predictiva	Adaptativa
El objetivo es claro y el proceso está definido	El cliente no conoce cuáles son exactamente sus necesidades
Planificación meticulosa	Planificación esbozada
Evita el cambio, el cambio no es bienvenido	Acepta el cambio, se buscan los cambios
Control y Coordinación	Inspección y Adaptación
Presupuestos precisos	Alcance abierto
Documentación extensa	Pocos artefactos
Silos de conocimiento	Equipo interdisciplinar

COMPARACIÓN ENTRE VISIONES

A pesar de que la visión adaptativa está ampliamente implantada en la industria del software, existen excepciones a tener en cuenta. En el caso de construir un software fuertemente acoplado a otros productos industriales o a dispositivos hardware, por ejemplo software embebido para equipamiento médico o software de trazabilidad de una línea de producción industrial o software controlador de un avión o satélite, pueden resultar adecuados los procesos de gestión predictiva.

- ❑ Maleabilidad del producto. La capacidad del software para ofrecer nuevas funcionalidades (extensibilidad) queda estrechamente ligada a la capacidad evolutiva de los dispositivos físicos al que el software va ligado.
- ❑ Factor de escala del producto. El factor de escala del producto software no ofrece valor al “acoplarse” el software al producto físico.

En estas circunstancias el principal valor que se espera de la metodología de construcción es la correcta **planificación**, el **software es solo uno más de los elementos del producto final**. Es imprescindible cumplir los hitos de entrega para coordinar el “ensamblaje” del software con el hardware o dispositivos físicos con los que debe comunicarse.

Asimismo, en estos escenarios los procesos del negocio a los que el software debe dar solución han sido ya muy aterrizados debido a la necesidad de la industria “manufacturera” de definir con detalle el producto físico al que se “acoplará” el software. **No es necesario “descubrir” el producto software, los requisitos vendrán impuestos por el producto físico**, las especificaciones del software pueden “recogerse al dictado”.

